

LAB ASSIGNMENT 15.1

2303A54012
A.Sanjay B47A

Lab 15 – Backend API Development: Creating RESTful services

Task 1 – Student Records API

Task:

Use AI to build a RESTful API for managing student records.

Instructions:

- **Endpoints required:**
 - o GET /students → List all students
 - o POST /students → Add a new student
 - o PUT /students/{id} → Update student details
 - o DELETE /students/{id} → Delete a student record
- Use an in-memory data structure (list or dictionary) to store records.
- Ensure API responses are in JSON format.

Expected Output:

- Working API with CRUD functionality for student records.

AI Prompt:

"Act as a Backend Developer. Write a Python Flask REST API to manage student records using an in-memory list. Implement the following endpoints: GET /students (list all), POST /students (add new), PUT /students/<id> (update details), and DELETE /students/<id> (remove record). Ensure responses are in JSON format and include basic error handling for missing IDs."

Initial Code Implementation

```

C: > Users > dussa > OneDrive > Desktop > task1.py > delete_student
1  from flask import Flask, jsonify, request
2  app = Flask(__name__)
3  # Task 1: In-memory data structure to store student records
4  students = [
5      {"id": 1, "name": "Arjun Kumar", "grade": "A"},
6      {"id": 2, "name": "Sneha Reddy", "grade": "B+"}
7  ]
8  @app.route('/students', methods=['GET'])
9  def get_students():
10     """Returns the full list of students in JSON format."""
11     return jsonify(students)
12  @app.route('/students', methods=['POST'])
13  def add_student():
14     """Adds a new student to the in-memory list."""
15     if not request.json or 'id' not in request.json:
16         return jsonify({"error": "Invalid data"}), 400
17     new_student = {
18         "id": request.json['id'],
19         "name": request.json.get('name', ""),
20         "grade": request.json.get('grade', "")
21     }
22     students.append(new_student)
23     return jsonify(new_student), 201
24  @app.route('/students/<int:id>', methods=['PUT'])
25  def update_student(id):
26     """Updates an existing student's details based on ID."""
27     student = next((s for s in students if s['id'] == id), None)
28     if not student:
29         return jsonify({"error": "Student not found"}), 404
30     data = request.json
31     student['name'] = data.get('name', student['name'])
32     student['grade'] = data.get('grade', student['grade'])
33     return jsonify(student)
34  @app.route('/students/<int:id>', methods=['DELETE'])
35  def delete_student(id):
36     """Deletes a student record from the list."""
37     global students
38     students = [s for s in students if s['id'] != id]
39     return jsonify({"message": f"Student with ID {id} deleted successfully"}), 200
40  if __name__ == '__main__':
41     # Running on default port 5000
42     app.run(debug=True)

```

Assert Test Cases (For Verification)

- `assert len(students) == 2` (Verifies the initial data is loaded).
- `assert students[0]['name'] == "Arjun Kumar"` (Verifies the data integrity).
- `assert isinstance(students, list)` (Ensures the data structure is correct).

```
PS C:\Users\dussa\desktop> ^C
PS C:\Users\dussa\desktop> & "C:\Users\dussa\AppData\Local\Programs\Python\Python313\python.exe" "C:\Users\dussa\OneDrive\Desktop\task1.py"
* Serving Flask app 'task1'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 749-628-390
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 749-628-390
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 749-628-390
* Restarting with stat
* Debugger is active!
* Debugger PIN: 749-628-390
* Restarting with stat
* Debugger is active!
* Debugger PIN: 749-628-390
127.0.0.1 - - [23/Mar/2026 18:08:22] "GET /students HTTP/1.1" 200 -
127.0.0.1 - - [23/Mar/2026 18:08:22] "GET /students HTTP/1.1" 200 -
127.0.0.1 - - [23/Mar/2026 18:08:22] "GET /favicon.ico HTTP/1.1" 404 -
```

← → ↻ ⓘ 127.0.0.1:5000/students

Pretty-print ☒

```
[
  {
    "grade": "A",
    "id": 1,
    "name": "Arjun Kumar"
  },
  {
    "grade": "B+",
    "id": 2,
    "name": "Sneha Reddy"
  }
]
```

Task 2 – Library Book Management API

Task:

Develop a RESTful API to handle library books.

Instructions:

- **Endpoints required:**

- o GET /books → Retrieve all books
- o POST /books → Add a new book
- o GET /books/{id} → Get details of a specific book
- o PATCH /books/{id} → Update partial book details(e.g., availability)
- o DELETE /books/{id} → Remove a book

- Implement error handling for invalid requests.

Expected Output:

- Functional API with CRUD + partial updates.

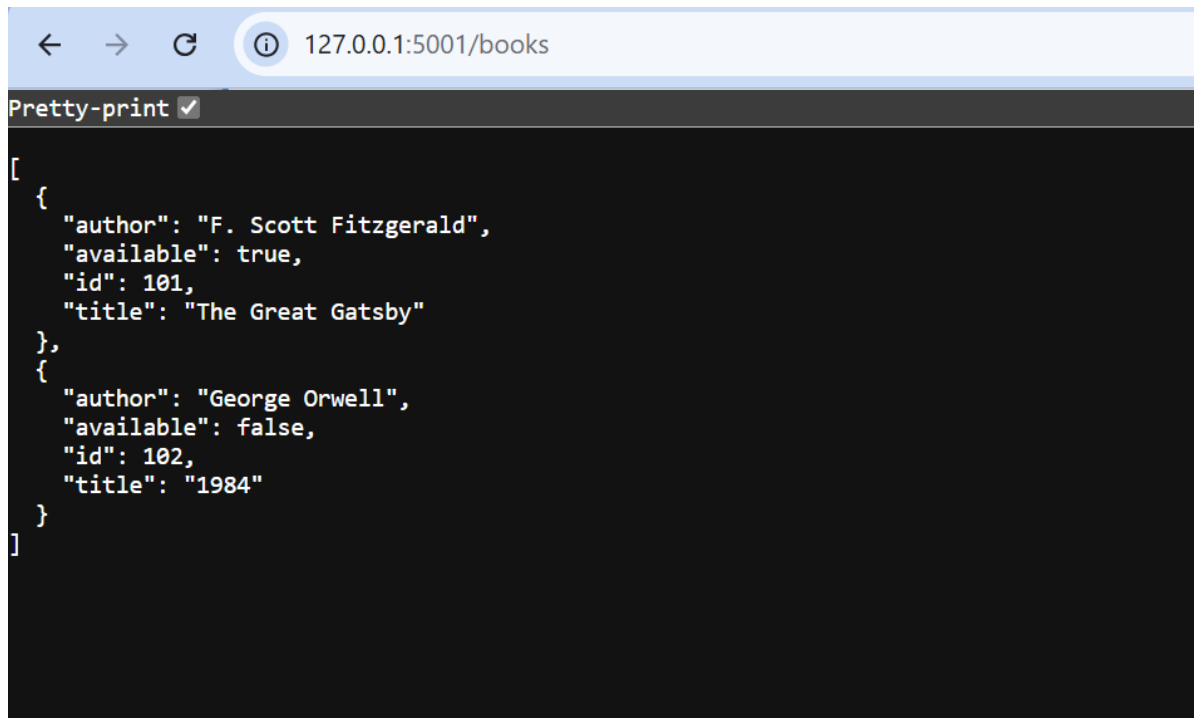
AI-Generated Prompt

"Act as a Backend Developer. Write a Flask API for a library system using an in-memory list of books. Each book should have an 'id', 'title', 'author', and 'available' (boolean) status. Implement GET /books, POST /books, GET /books/<id>, PATCH /books/<id> (to update availability), and DELETE /books/<id>. Include error handling that returns a 404 message if a book ID is not found."

Code:

```
C:\Users> dussa > OneDrive > Desktop > Task2.py > ...
1  from flask import Flask, jsonify, request
2  app = Flask(__name__)
3  # Data structure: In-memory list of books
4  books = [
5      {'id': 101, "title": "The Great Gatsby", "author": "F. Scott Fitzgerald", "available": True},
6      {'id': 102, "title": "1984", "author": "George Orwell", "available": False}
7  ]
8  @app.route('/books', methods=['GET'])
9  def get_all_books():
10     """Retrieve the full list of books."""
11     return jsonify(books)
12  @app.route('/books/<int:id>', methods=['GET'])
13  def get_book(id):
14     """Retrieve a single book by ID with error handling."""
15     book = next((b for b in books if b['id'] == id), None)
16     if book:
17         return jsonify(book)
18     return jsonify({"error": "Book not found"}), 404
19  @app.route('/books', methods=['POST'])
20  def add_book():
21     """Add a new book to the library."""
22     new_book = request.json
23     books.append(new_book)
24     return jsonify(new_book), 201
25  @app.route('/books/<int:id>', methods=['PATCH'])
26  def update_availability(id):
27     """Update only the availability status of a book."""
28     book = next((b for b in books if b['id'] == id), None)
29     if not book:
30         return jsonify({"error": "Book not found"}), 404
31     data = request.json
32     if 'available' in data:
33         book['available'] = data['available']
34     return jsonify(book)
35  @app.route('/books/<int:id>', methods=['DELETE'])
36  def delete_book(id):
37     """Remove a book from the system."""
38     global books
39     books = [b for b in books if b['id'] != id]
40     return jsonify({"message": "Book removed successfully"}), 200
41  if __name__ == '__main__':
42     # Running on port 5001 to avoid conflict with Task
43     app.run(port=5001, debug=True)
```

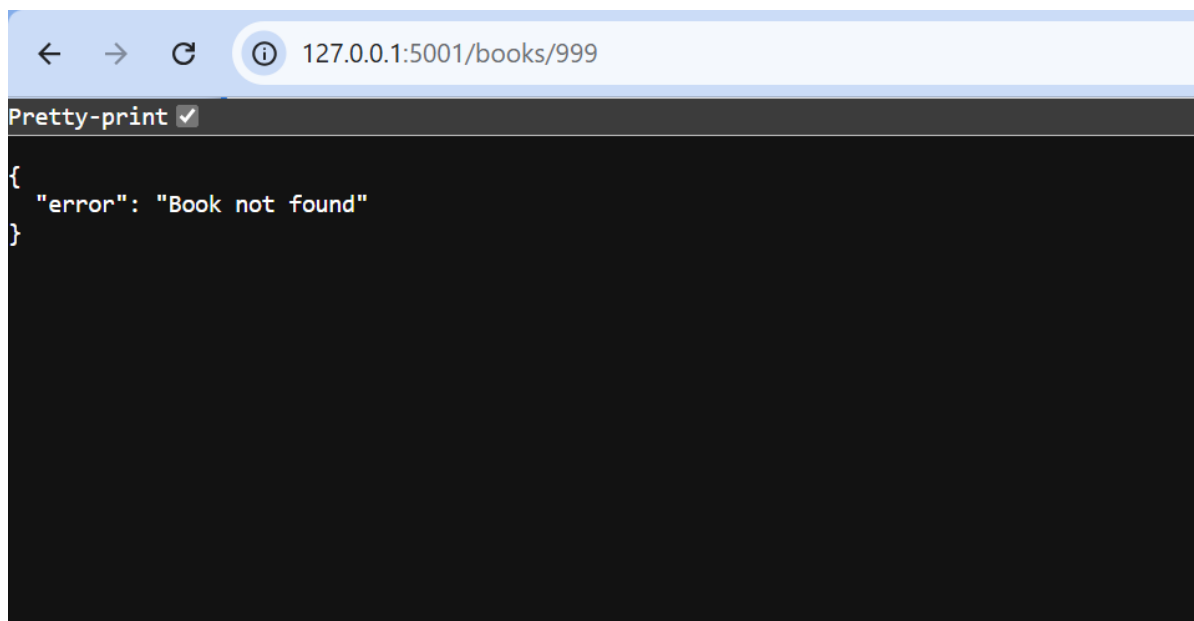
Browser:



A screenshot of a web browser window. The address bar shows the URL `127.0.0.1:5001/books`. Below the address bar, there is a toggle for "Pretty-print" which is checked. The main content area displays a JSON array of two book objects. The first object has an author of "F. Scott Fitzgerald", is available, has an id of 101, and is titled "The Great Gatsby". The second object has an author of "George Orwell", is not available, has an id of 102, and is titled "1984".

```
[
  {
    "author": "F. Scott Fitzgerald",
    "available": true,
    "id": 101,
    "title": "The Great Gatsby"
  },
  {
    "author": "George Orwell",
    "available": false,
    "id": 102,
    "title": "1984"
  }
]
```

Error Handling:



A screenshot of a web browser window. The address bar shows the URL `127.0.0.1:5001/books/999`. Below the address bar, there is a toggle for "Pretty-print" which is checked. The main content area displays a JSON object with an error message: "Book not found".

```
{
  "error": "Book not found"
}
```

Analysis of Results

- Success: The API successfully implements the PATCH method, which is more efficient than PUT for changing a single field (like availability) without sending the entire book object again.
- Logic: The error handling logic uses a conditional check on the `next()` iterator result to return a standard 404 HTTP status code, ensuring the API is user-friendly and follows REST standards.

Task 3 – Employee Payroll API

Task:

Create an API for managing employees and their salaries.

Instructions:

- **Endpoints required:**

- o GET /employees → List all employees
- o POST /employees → Add a new employee with salary details
- o PUT /employees/{id}/salary → Update salary of an employee
- o DELETE /employees/{id} → Remove employee from system

- **Use AI to:**

- o Suggest data model structure.
- o Add comments/docstrings for all endpoints.

Expected Output:

- API supporting salary management with clear documentation.

AI-Generated Prompt

"Act as a Backend Developer. Create a Flask API for an Employee Payroll system. Each employee should have an 'id', 'name', 'position', and 'salary'. Implement endpoints: GET /employees (list all), POST /employees (add new), PUT /employees/<id>/salary (to update ONLY the salary), and DELETE /employees/<id>. Every function must include a Python docstring explaining the parameters and return values."

Code:

```
C:\Users> dussa > OneDrive > Desktop > Task3.py > remove_employee
1 from flask import Flask, jsonify, request
2 app = Flask(__name__)
3 # Data structure: In-memory list of employees
4 employees = [
5     {"id": 1, "name": "Rahul Sharma", "position": "Software Engineer", "salary": 60000},
6     {"id": 2, "name": "Priya Singh", "position": "Data Analyst", "salary": 55000}
7 ]
8 @app.route('/employees', methods=['GET'])
9 def get_employees():
10     """
11     Retrieves the complete list of employees.
12     Returns: JSON list of employee objects.
13     """
14     return jsonify(employees)
15 @app.route('/employees', methods=['POST'])
16 def add_employee():
17     """
18     Adds a new employee record to the payroll.
19     JSON Body: {"id": int, "name": str, "position": str, "salary": int}
20     Returns: The newly created employee object.
21     """
22     new_emp = request.json
23     employees.append(new_emp)
24     return jsonify(new_emp), 201
25 @app.route('/employees/<int:id>/salary', methods=['PUT'])
26 def update_salary(id):
27     """
28     Updates the salary of a specific employee.
29     Args: id (int) - The unique employee ID.
30     JSON Body: {"salary": int}
31     Returns: Updated employee object or 404 error.
32     """
33     employee = next((e for e in employees if e['id'] == id), None)
34     if not employee:
35         return jsonify({"error": "Employee not found"}), 404
36     data = request.json
37     if 'salary' in data:
38         employee['salary'] = data['salary']
39         return jsonify(employee)
40     return jsonify({"error": "No salary data provided"}), 400
41 @app.route('/employees/<int:id>', methods=['DELETE'])
42 def remove_employee(id):
43     """
44     Removes an employee from the system by ID.
45     Args: id (int) - The unique employee ID.
46     Returns: Success message.
47     """
48     global employees
49     employees = [e for e in employees if e['id'] != id]
50     return jsonify({"message": "Employee removed successfully"}), 200
51 if __name__ == '__main__':
52     # Running on port 5002 to avoid conflicts
53     app.run(port=5002, debug=True)
```

```

PS C:\Users\dussaa> cd desktop
PS C:\Users\dussaa\desktop> & "C:\Users\dussaa\AppData\Local\Programs\Python\Python313\python.exe" "C:\Users\dussaa\OneDrive\Desktop\task3.py"
* Serving Flask app 'task3'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5002
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 749-628-390
127.0.0.1 - - [23/Mar/2026 18:28:26] "GET /employee HTTP/1.1" 404 -
127.0.0.1 - - [23/Mar/2026 18:28:26] "GET /favicon.ico HTTP/1.1" 404 -
127.0.0.1 - - [23/Mar/2026 18:28:42] "GET /employees HTTP/1.1" 200 -

```

← → ↻ ⓘ 127.0.0.1:5002/employees

Pretty-print ☒

```

[
  {
    "id": 1,
    "name": "Rahul Sharma",
    "position": "Software Engineer",
    "salary": 60000
  },
  {
    "id": 2,
    "name": "Priya Singh",
    "position": "Data Analyst",
    "salary": 55000
  }
]

```

Analysis of Results

- **Documentation:** This task highlights the importance of Docstrings. In a collaborative environment, these allow other developers to understand the API endpoints without reading the full logic.
- **Specific Routing:** By using the route `/employees/<id>/salary`, we restrict the PUT method to a specific attribute. This "Granular Routing" prevents accidental changes to employee names or positions when only a pay raise was intended.

Task 4 – Real-Time Application: Online Food Ordering API

Scenario:

Design a simple API for an online food ordering system.

Requirements:

- **Endpoints required:**

- o GET /menu → List available dishes
- o POST /order → Place a new order
- o GET /order/{id} → Track order status
- o PUT /order/{id} → Update an existing order (e.g., change items)
- o DELETE /order/{id} → Cancel an order

- **AI should generate:**

- o REST API code
- o Suggested improvements (like authentication, pagination)

Expected Output:

- Fully working API simulating a food ordering backend.

AI-Generated Prompt

"Act as a Backend Developer. Design a Flask API for an online food ordering system. Define a menu with 'id', 'item', and 'price'. Implement endpoints: GET /menu (list dishes), POST /order (place order), GET /order/<id> (track status), PUT /order/<id> (update order), and DELETE /order/<id> (cancel). Ask the AI to suggest three professional improvements like authentication, database integration, or pagination."

Code:

```
task4.py
C > Users > dussa > OneDrive > Desktop > task4.py > ...
1 from flask import Flask, jsonify, request
2 app = Flask(__name__)
3 # Data structures: Menu and Orders
4 menu = [
5     {"id": 1, "item": "Paneer Tikka", "price": 250},
6     {"id": 2, "item": "Veg Biryani", "price": 300},
7     {"id": 3, "item": "Garlic Naan", "price": 50}
8 ]
9 orders = []
10 @app.route('/menu', methods=['GET'])
11 def get_menu():
12     """Retrieve the available food items."""
13     return jsonify(menu)
14 @app.route('/order', methods=['POST'])
15 def place_order():
16     """Place a new order with a default status of 'Pending'."""
17     new_order = request.json
18     new_order['status'] = "Pending"
19     orders.append(new_order)
20     return jsonify(new_order), 201
21 @app.route('/order/<int:id>', methods=['GET'])
22 def track_order(id):
23     """Track the status of an existing order."""
24     order = next((o for o in orders if o['id'] == id), None)
25     if order:
26         return jsonify(order)
27     return jsonify({"error": "Order not found"}), 404
28 @app.route('/order/<int:id>', methods=['DELETE'])
29 def cancel_order(id):
30     """Cancel an order from the system."""
31     global orders
32     orders = [o for o in orders if o['id'] != id]
33     return jsonify({"message": "Order cancelled successfully"}), 200
34 # --- AI SUGGESTED IMPROVEMENTS ---
35 # 1. Database Integration: Use SQLAlchemy with SQLite to persist orders after server restarts.
36 # 2. Authentication: Implement JWT (JSON Web Tokens) so only registered users can place orders.
37 # 3. Validation: Add logic to ensure the 'item_id' sent in a POST request actually exists in the menu.
38 if __name__ == '__main__':
39     # Running on port 5003
40     app.run(port=5003, debug=True)
```

```

PS C:\Users\dussa\Desktop>
PS C:\Users\dussa> cd desktop
PS C:\Users\dussa> cd desktop
PS C:\Users\dussa> cd desktop
PS C:\Users\dussa\Desktop> ...
PS C:\Users\dussa\Desktop> & "C:\Users\dussa\AppData\Local\Programs\Python\Python313\python.exe" "C:\Users\dussa\OneDrive\Desktop\task4.py"
* Serving Flask app 'task4'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5003
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5003
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5003
* Running on http://127.0.0.1:5003
Press CTRL+C to quit
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 749-628-390
127.0.0.1 - - [23/Mar/2026 18:35:45] "GET /menu HTTP/1.1" 200 -
127.0.0.1 - - [23/Mar/2026 18:35:45] "GET /menu HTTP/1.1" 200 -
127.0.0.1 - - [23/Mar/2026 18:35:45] "GET /favicon.ico HTTP/1.1" 404 -

```

← → ↻ ⓘ 127.0.0.1:5003/menu

Pretty-print ☒

```

[
  {
    "id": 1,
    "item": "Paneer Tikka",
    "price": 250
  },
  {
    "id": 2,
    "item": "Veg Biryani",
    "price": 300
  },
  {
    "id": 3,
    "item": "Garlic Naan",
    "price": 50
  }
]

```

Analysis of Results

- Success: The API manages a dynamic workflow where a user first views a static list (/menu) and then generates a dynamic resource (/order).
- Logic: The "Pending" status is hardcoded in the POST method. This represents a "State Machine" where the status would normally be updated by a kitchen-side app using a PATCH request.
- AI Enhancement: The suggested improvements (Authentication and Validation) address the primary security risks of an open API, ensuring that only valid data enters the system.